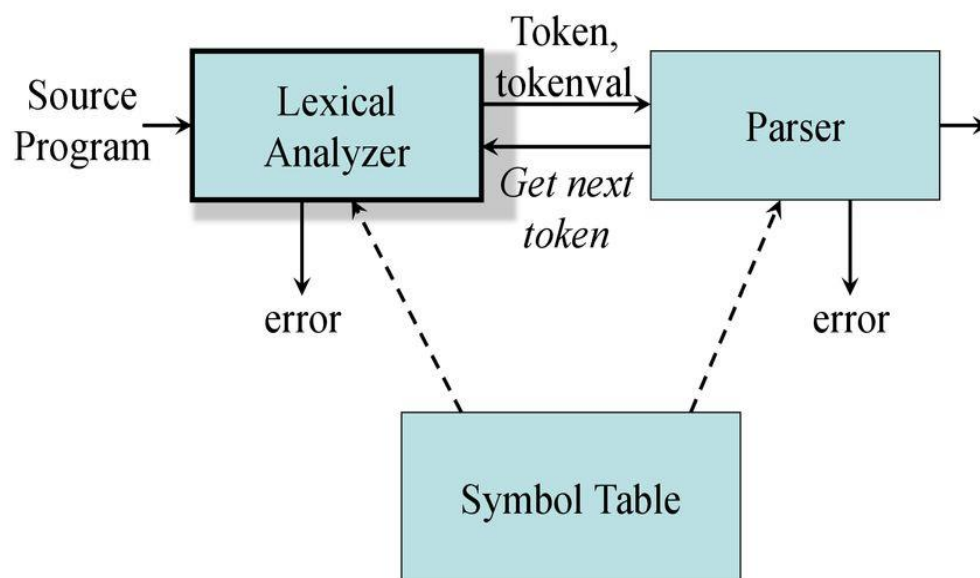


Chapter 4 : Lexical and syntax analysis

Prepared by: Assoc. Prof. Rasha H. Sakr

3

Interaction of the Lexical Analyzer with the Parser



Lexical analyzer

- A **lexical analyzer** collects input characters into groups (**lexemes**) and assigns an internal code (a **token**) to each group.
- **Lexemes** are sequences of characters forming meaningful units
- **Tokens** are categorized lexemes with assigned types (keyword, identifier, etc.)
- **Patterns** are rules that the lexical analyzer uses to identify lexemes and classify them into tokens. (often expressed regular expressions) used to identify and classify lexemes into tokens. A regular expression defines a set of strings that match a specific pattern, allowing the lexical analyzer to recognize valid tokens from the source code. For example, a pattern might define an identifier as a letter followed by zero or more letters or digits.
- Tokens are usually coded as integer values, but for the sake of readability, they are often referenced through named constants.
- **An example assignment statement:**

```
result = oldsum - value / 100;
```

Tokens and lexemes of this statement:

<i>Token</i>	<i>Lexeme</i>
IDENT	result
ASSIGN_OP	=
IDENT	oldsum
SUB_OP	-
IDENT	value
DIV_OP	/
INT_LIT	100
SEMICOLON	;

- In early compilers, lexical analyzers often processed an entire program

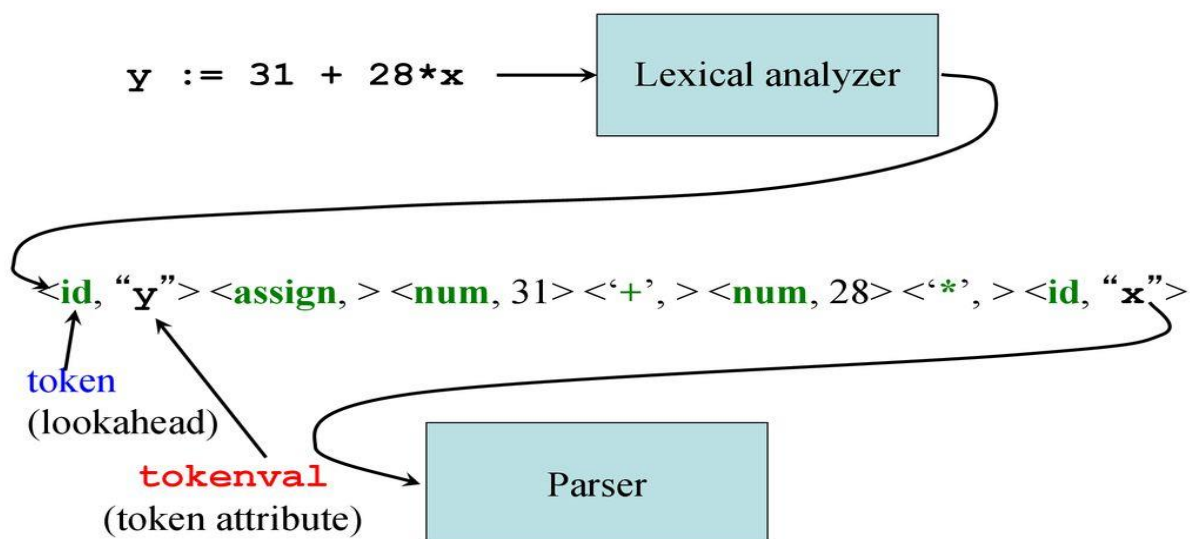
file and produced a file of tokens and lexemes. Now, most lexical analyzers are subprograms that return the next lexeme and its associated token code when called.

- Other tasks performed by a lexical analyzer:

- Skipping comments and white space between lexemes.
- Inserting lexemes for user-defined names into the symbol table.
- Detecting syntactic errors in tokens, such as ill-formed floating-point literals.

4

Attributes of Tokens



Lexeme

A lexeme is a sequence of characters of a program that is grouped together as a single unit. When a compiler or interpreter reads the source code of a program, the compiler breaks it down into smaller units called lexemes. These lexemes will help the compiler to analyze and process the program efficiently.

Lexemes are assigned to a token type, which is a numeric code that represents the type of lexemes. These tokens can be passed to the parser, which can be used to construct the parse tree with various methods of parsing.

For example, If there is a statement 'a = b / 2;' in C++ programming language. This statement will be broken into:

- **Identifier:** 'a'
- **Operator:** '='
- **Identifier:** 'b'
- **Operator:** '/'
- **Literal:** '2'
- **Punctuation:** ';'.

Categories of Lexemes

1. **Operators:** These are the symbols that can be used to perform mathematical or logical operations such as addition (+), subtraction (-), multiplication (*), division (/), etc.
2. **Identifiers:** These are the name of variables, methods, and functions. For example, If there is a statement 'int a = 5;' in C++. Here 'a' is an Identifier.
3. **Punctuation:** These are the symbols that are used to separate and group different parts of the program. For example, semicolon (;), comma (,), braces ({}), etc.
4. **Keywords:** These are used to identify programming constructs such as loops, conditionals, and functions. For example, 'if', 'else', 'while', etc.
5. **Literals:** These are the values that can be used in the source code. For example, If there is a statement 'int a = 5;' in C++. Here '5' is literal. Literals can be of any given data type.

Calculating Lexemes in Program

Even though the calculation of lexemes in a program is done by the [lexers](#) but to understand lexemes more, Let's take an example:

Consider the following C++ program:

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    cout << "The value of x is: " << x << endl;
    return 0;
}
```

The total number of lexemes in the above program is **29 lexemes**.

To count the number of in this program, we need to break it down into constituent parts (or each line) and identify and calculate the lexemes in each part:

```
include <iostream> // 3 lexemes: "include", "<iostream>", and ">"
using namespace std; // 4 lexemes: "using", "namespace", "std", and ";"

int main() { // 3 lexemes: "int", "main", and "("
    int x = 10; // 5 lexemes: "int", "x", "=", "10", and ";"
    cout << "The value of x is: " << x << endl; // 11 lexemes: "cout", "<<", ""The value of x is: "",
    "<<", "x", "<<", "endl", and ";"
    return 0; // 3 lexemes: "return", "0", and ";"
}
```

Frequently Asked Questions

Why Lexemes are important in Compiler Design?

Lexemes are used to understand and analyze the source code by the compiler. Lexemes are used in breaking down the source code into smaller units to process the program. This makes the process of Understanding and Analyzing easier and more efficient.

How lexeme can be identified?

Here are 6 categories of lexemes that can be present in the program; we identify these lexemes by these categories :

1. **Operators:** +, -, /, *, and %.
2. **Identifiers:** If there is a statement 'int a = 5' in C++. Here 'a' is an Identifier.
3. **Punctuation:** semicolon (;), comma (,), braces ({}), etc.
4. **Keywords:** 'if', 'else', 'while', etc.
5. **Literals:** 5, 3.14, 89, "Ninja", etc.

Why lexemes are used?

Lexemes will help the compiler to analyze and process the program efficiently. Each Lexeme is assigned a token type, which is a numeric code that represents the type of lexeme. These tokens can be passed to the parser, which can be used to construct the parse tree with various methods of parsing.

What is Lexer?

Lexer (or lexical analyzer) is the first component in the compilation process which produces lexeme. Lexical Analyzer reads the source code character by character and

groups them as a single unit, and then these lexemes can be passed to the parser for constructing the parsing tree.

Conclusion

A lexeme is a sequence of characters of a program that is grouped together as a single unit. When a compiler or interpreter reads the source code of a program, the compiler breaks it down into smaller units called lexemes. In this article, we discussed what lexemes are, how lexemes can be identified, and the types of lexemes, and we also discussed an example of calculating lexemes in a C++ program.

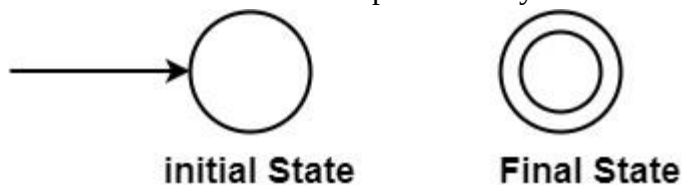
Lexical Analysis can be designed using Transition Diagrams.

- A [state transition diagram](#), or **state diagram**, is a -
directed graph. The nodes are labeled with state names.
The arcs are labeled with input characters.
An arc may also include actions to be done when the transition is taken.

Finite Automata (Transition Diagram) – A Directed Graph or flowchart used to recognize token.

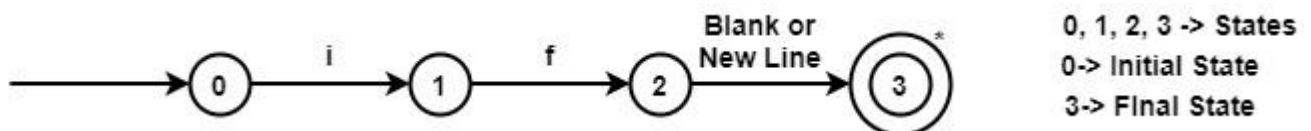
The transition Diagram has two parts –

- **States** – It is represented by circles.

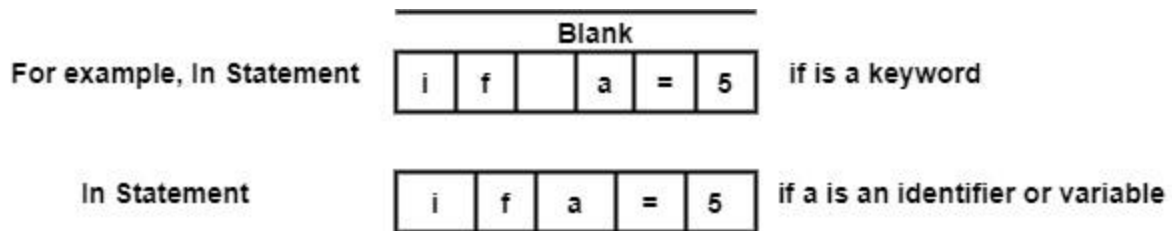


- **Edges** – States are connected by Edges Arrows.

Example – Draw Transition Diagram for "if" keyword.



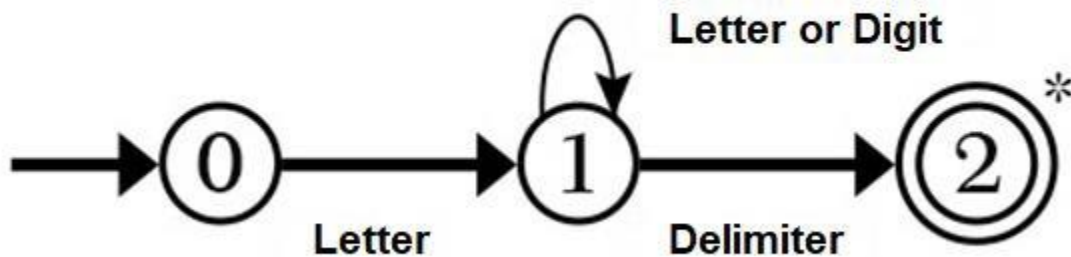
To recognize Token ("if"), Lexical Analysis has to read also the next character after "f". Depending upon the next character, it will judge whether the "if" keyword or something else is.



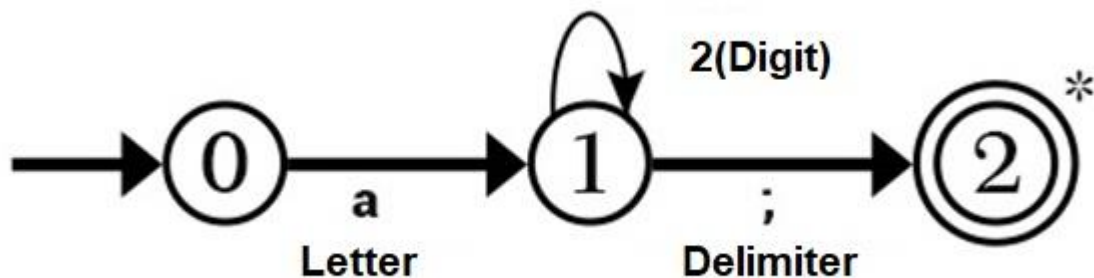
So, Blank space after "if" determines that "If" is a keyword.

"*" on Final State 3 means Retract, i.e., control will again come to previous state 2. Therefore Blank space is not a part of the Token ("if").

Transition Diagram for an Identifier – An identifier starts with a letter followed by letters or Digits. Transition Diagram will be:



For example, In statement **int a2;** Transition Diagram for identifier **a2** will be:



As (;) is not part of Identifier ("a2"), so use "*" for Retract i.e., coming back to state 1 to recognize identifier ("a2").

The Transition Diagram for identifier can be converted to Program Code as –

Coding

State 0: C = `Getchar()`

If `letter` (C) then `goto` state 1 else fail

```

State1: C = Getchar()
If letter (C) or Digit (C) then goto state 1
else if Delimiter (C) goto state 2
else Fail

State2: Retract ()
return (6, Install ());

```

In-state 2, Retract () will take the pointer one state back, i.e., to state 1 & declares that whatever has been found till state 1 is a token.

The lexical Analysis will return the token to the Parser, not in the form of an English word but the form of a pair, i.e., (Integer code, value).

In the case of identifier, the integer code returned to the parser is 6 as shown in the table.

Install () – It will return a pointer to the symbol table, i.e., address of tokens.

The following table shows the integer code and value of various tokens returned by lexical analysis to the parser.

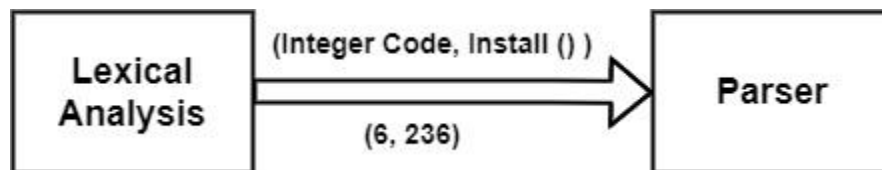
Integer Codes for different Tokens

Lexeme / Token	Integer Code	Value
Begin	1	-
End	2	-
If	3	-
Then	4	-
Else	5	-
Identifier	6	Pointer to Symbol Table
Constants	7	Pointer to Symbol Table
<	8	1
<=	8	2
=	8	3

Lexeme / Token	Integer Code	Value
<>	8	4
>	8	5
>=	8	6

These integer values are not fixed. Different Programmers can choose other integer codes and values while designing the Lexical Analysis.

Suppose, if the **identifier** is stored at location 236 in the symbol table, then



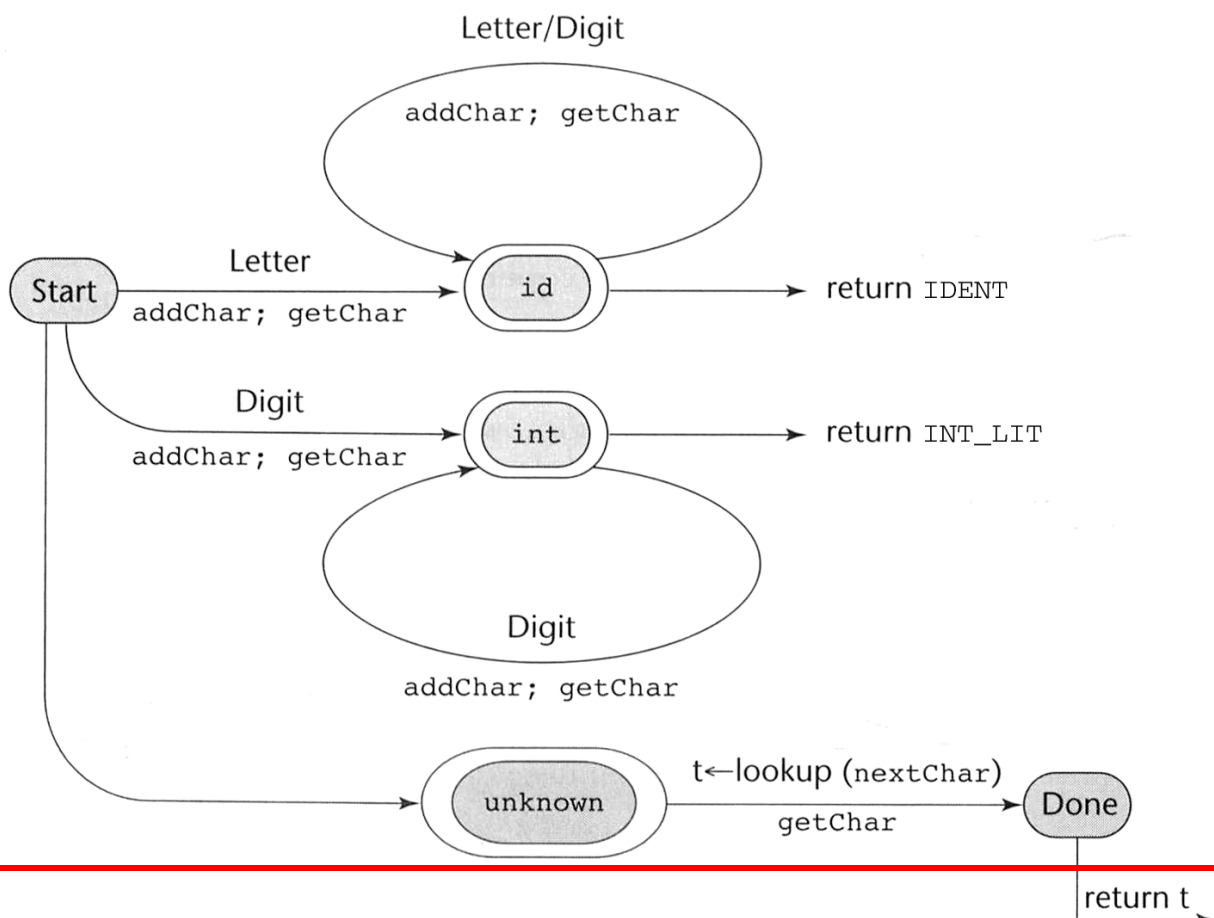
Similarly, if **constant** is stored at location 238 then

Integer code = 7

Install () = 238 i.e., Pair will be (7, 238)

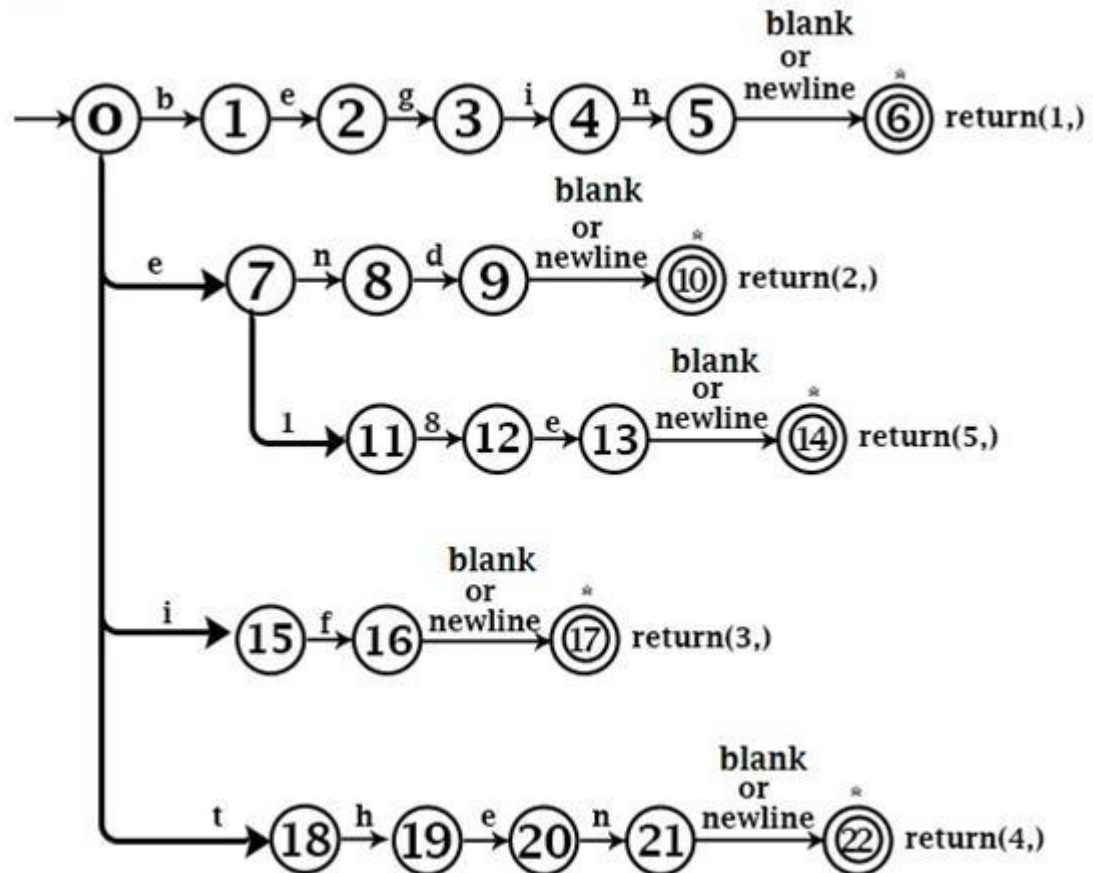
Transition Diagram (Finite Automata) for Tokens –

- A state diagram that recognizes names, integer literals, parentheses, and arith-metic operators:



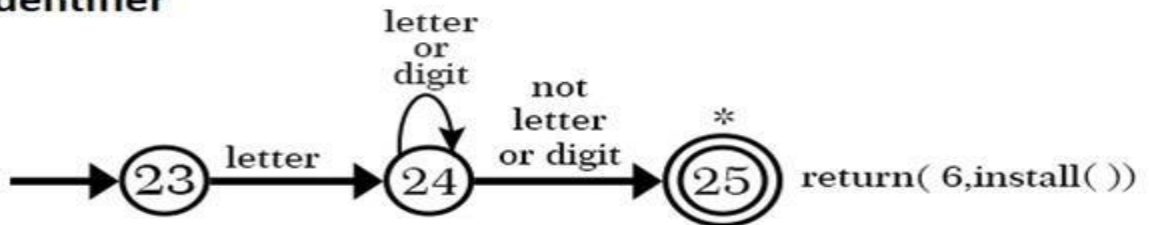
The diagram includes the actions required on each transition.

Keywords



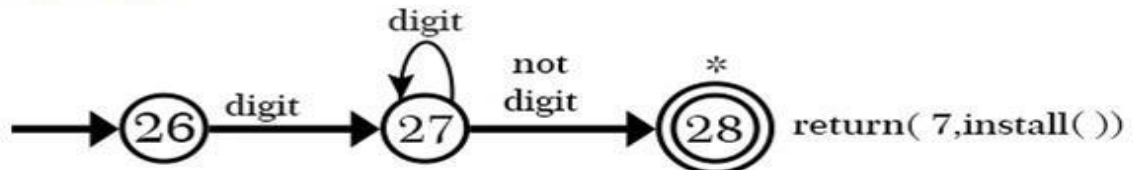
Transition Diagram for Keywords

Identifier



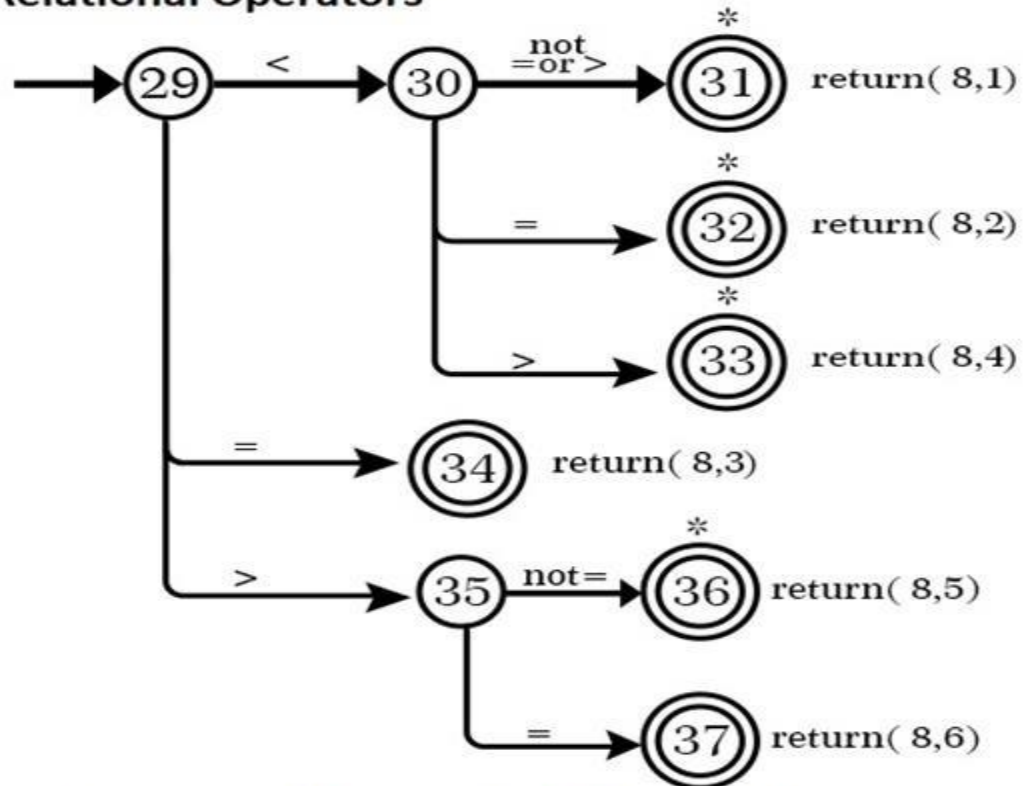
Transition Diagram for Identifier

Constant



Transition Diagram for Constant

Relational Operators



Transition Diagram for Relational Operators

Introduction to Parsing

- Syntax analysis is often referred to as **parsing**.
- Responsibilities of a syntax analyzer, or parser:
Determine whether the input program is syntactically correct.
Produce a parse tree. In some cases, the parse tree is only implicitly constructed.
- When an error is found, a parser must produce a diagnostic message and recover. Recovery is required so that the compiler finds as many errors as possible.

- Parsers are categorized according to the direction in which they build parsetrees:

Top-down parsers build the tree from the root downward to the leaves.

Bottom-up parsers build the tree from the leaves upward to the root.

- Notational conventions for grammar symbols and strings:

Terminal symbols—Lowercase letters at the beginning of the alphabet (a, b, ...)

Nonterminal symbols—Uppercase letters at the beginning of the alphabet (A, B, ...)

Terminals or nonterminals—Uppercase letters at the end of the alphabet (W, X, Y, Z)

Strings of terminals—Lowercase letters at the end of the alphabet (w, x, y, z)

Mixed strings (terminals and/or nonterminals)—Lowercase Greek letters (α , β , γ , δ)

Top-Down Parsers

- A top-down parser traces or builds the parse tree in preorder: each node is visited before its branches are followed.
- The actions taken by a top-down parser correspond to a leftmost derivation.

Bottom-Up Parsers

- A bottom-up parser constructs a parse tree by beginning at the leaves and progressing toward the root. This parse order corresponds to the reverse of a rightmost derivation.
- Given a right sentential form α , a bottom-up parser must determine what substring of α is the right-hand side (RHS) of the rule that must be reduced to its LHS to produce the previous right sentential form.
- Bottom-up parsers are often called **shift-reduce algorithms**, because shift and reduce are their two fundamental actions.

The shift action moves the next input token onto the parser's stack.
A reduce action replaces a RHS (the handle) on top of the parser's stack by its corresponding LHS.

LR Parsing

Introduction

The parser is a [compiler](#) phase that accepts a token string as input and turns it into the matching intermediate representation using existing grammar. A parser is also known as a [syntax analyzer](#). There are two types of parsing techniques in compiler design: Top-down parsing and Bottom-up parsing.

- **Top-Down Parsing**- It is a parsing strategy that starts at the top of the parse tree and works its way down by applying grammatical rules.

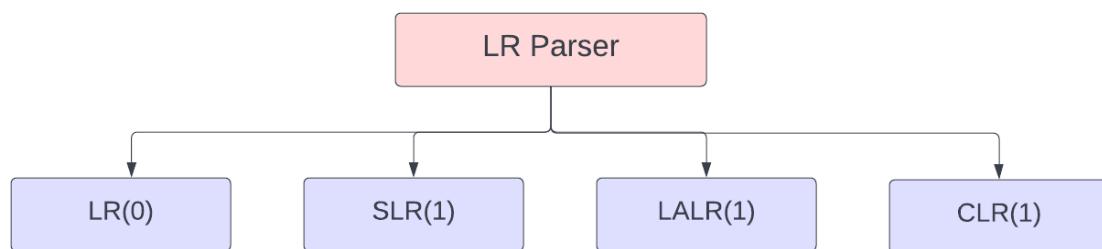
- **Bottom-up Parsing-** It is a parsing strategy that begins with the lowest level of the parse tree and works its way up using grammatical rules.

This chapter will learn a type of bottom-up parsing, i.e., LR parsing.

LR Parsing

LR Parsing is a type of bottom-up parsing. It is used to parse context-free grammar, primarily used by compilers. The LR parser reads the input from left to right and produces the rightmost derivation.

The LR parsing can be further divided into four types-



The LR parser is denoted as LR(k), where the letter “L” refers to the left to right scanning of the input, the letter “R” refers to the rightmost derivation in reverse, and “k” refers to the number of unconsumed “look ahead” input symbols that are used in making parser decisions. The letter k is usually one and is frequently omitted. A context-free grammar is called LR(k) if the LR(k) parser exists.

- Advantages of LR parsers:

They can be built for all programming languages.

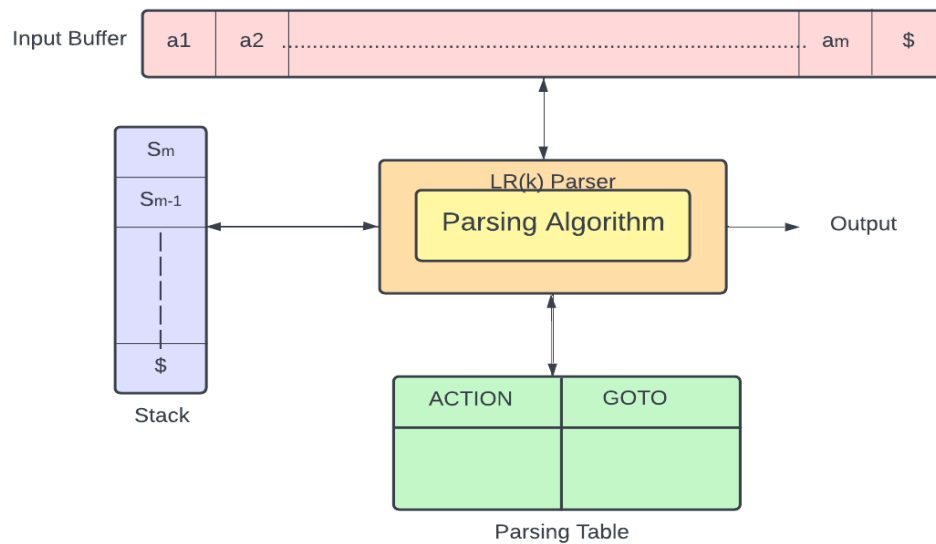
They can detect syntax errors as soon as possible in a left-to-right scan.

The LR class of grammars is a proper superset of the class parsable by LL parsers.

- It is difficult to produce an LR parsing table by hand. However, there are many programs available that take a grammar as input and produce the parsing table.

Next, we will discuss the block diagram of LR parsing.

Block Diagram



The block diagram of the LR parser.

The input buffer takes one symbol at a time from left to right $a_1, a_2, \dots, a_m, \$$. The stack contains the combination of the state symbol and the current input symbol. The parsing table is a two-dimensional array containing the Action and GoTo tables. Next, we will discuss the Parsing Table.

- The contents of the parse stack for an LR parser has the following form, where the S s are state symbols and the X s are grammar symbols:

$S_0X_1S_1X_2S_2\dots X_mS_m$ (top)

- An LR parser configuration is a pair of strings representing the stack and the remaining input:

$(S_0X_1S_1X_2S_2\dots X_mS_m, a_ia_{i+1}\dots a_n\$)$

The dollar sign is an end-of-input marker.

- The LR parsing process is based on the parsing table, which has two parts, ACTION and GOTO.
- The ACTION part has state symbols as its row labels and terminal symbols as its column labels.

- The parse table specifies what the parser should do, based on the state symbol on top of the parse stack and the next input symbol.
- The two primary actions are *shift* (shift the next input symbol onto the stack) and *reduce* (replace the handle on top of the stack by the LHS of the matching rule).
- Two other actions are possible: *accept* (parsing is complete) and *error* (a syntax error has been detected).
- The values in the GOTO part of the table indicate which state symbol should be pushed onto the parse stack after a reduction has been completed.

The row is determined by the state symbol on top of the parse stack after the handle and its associated state symbols have been removed. The column is determined by the LHS of the rule used in the reduction.

Parsing Table

The parsing table is a two-dimensional array consisting of two tables- the ACTION table and the GOTO table.

Let's see them one by one.

ACTION Table

The Action table specifies the grammar rules for implementing the input stream's current state and terminal. There are four cases used in the Action table. These are-

1. Shift- This operation involves moving the current symbol and the current state **n** from the input buffer onto the stack. So, it becomes the new present state.
2. Reduce-
 1. The number **m** is written to the output stream.
 2. The state is deleted from the stack, according to the symbol **m** mentioned on the left-hand side of rule **m**.
 3. The symbol **m** mentioned on the left-hand side of rule **m** indicates that a new state is looked up in the goto table and pushed onto the stack as the new current state.

3. Accept- After repeating the shift and reduce operations, if the stack contains the starting symbol of the input string and the input buffer is empty, i.e., includes the \$ symbol, the input string is said to be accepted.
4. Error- If the parser cannot perform the shift or the reduce operation, also the string is not accepted, then it is said to be in the error state.

This function is denoted by (S_m, a_i) , where S_m represents the four cases defined above and a_i the current input symbol.

GOTO Table

The GOTO Table in the parsing table tells the next state where the parsing should proceed.

This function is denoted by (S_m, a_i) , where S_m represents the four cases defined above and a_i the current input symbol.

Next, we will discuss the steps of LR parsing.

Steps

Initially put 0 in the stack and the input string with the \$ symbol in the input buffer. Then follow the following steps to process the LR parsing-

1. If the parsing table consists of the $ACTION[S_m, a_i]$ function equal to shift s , execute the shift operation. Push the current input symbol a_i and the next state $S = GOTO[S_m, a_i]$ to the stack. The symbol a_{i+1} will become the new current input symbol.
2. If the parsing table consists of the $ACTION[S_m, a_i]$ function equals to reduce $A \rightarrow B$, then execute the reduce operation, entering the configuration $(S_0S_1 \dots S_{m-r}S, a_ia_{i+1} \dots a_n\$)$ where r is the length of the right side of the production and $S = GOTO[S_m-r, A]$.
The parser will pop ' $2r$ ' symbols from the stack to expose S_{m-r} , then push both ' A ' (left side of production) and ' S ' (entry for $ACTION[S_m-r, A]$) onto the stack.
Note: The current input symbol is not changed in the reduce operation.
3. If the parsing table consists of the $ACTION[S_m, a_i]$ function equal to accept, then the parsing is said to be completed, and the input string is accepted.
4. If the parsing table consists of the $ACTION[S_m, a_i]$ function equals to error means the parsing has encountered an error. So, the error routine will be called.

Now, we will discuss an example to understand the LR parsing.

Example

Question: Consider the following grammar

$E \rightarrow E + T$
 $E \rightarrow T$
 $T \rightarrow T \times F$
 $T \rightarrow F$
 $F \rightarrow (E)$
 $F \rightarrow id$

The ACTION and GOTO functions of the LR parsing table is given as follows:

STATE	ACTION						GOTO		
	id	+	x	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				A			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Using the ACTION and GOTO function table, parse the input string “idxid+id”.

Solution:

First, we will label the given grammar.

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T \times F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

Second, we will take a stack and input buffer and put 0 (initial state) and the input string in them respectively.

	Stack	Symbol	Input Buffer	Parsing Action
(1)	0		idxid+id\$	

At the initial state 0, we have s5 for id in the ACTION function of the parsing table. So, we will push id to the symbol column and 5 to the stack.

	Stack	Symbol	Input Buffer	Parsing Action
(1)	0		idxid+id\$	Shift
(2)	0 5	id	xid+id\$	

At state 5, we have r6 operation. So, we will reduce id by $F \rightarrow id$ and pop 5 out of the stack. We will also push 3 to the stack since for state 0 (now the symbol at the top of the stack), the GOTO function says to go to state 3 for F.

	Stack	Symbol	Input Buffer	Parsing Action
(1)	0		idxid+id\$	Shift
(2)	0 5	id	xid+id\$	Reduce $F \rightarrow id$
(3)	0 3	F	xid+id\$	

At state 3, we have the r4 operation for the symbol. So, we will reduce F by $T \rightarrow F$ and pop 3 out of the stack. We will also push 2 to the stack since for state 0 (now the symbol at the top of the stack), the GOTO function says to go to state 2 for T.

	Stack	Symbol	Input Buffer	Parsing Action
(1)	0		idxid+id\$	Shift
(2)	0 5	id	xid+id\$	Reduce $F \rightarrow id$
(3)	0 3	F	xid+id\$	Reduce $T \rightarrow F$
(4)	0 2	T	xid+id\$	

At state 2, we have the s7 operation for symbol x. So, we will push x to the symbol column and 7 to the stack.

	Stack	Symbol	Input Buffer	Parsing Action
--	-------	--------	--------------	----------------

(1)	0		idxid+id\$	Shift
(2)	0 5	id	xid+id\$	Reduce $F \rightarrow id$
(3)	0 3	F	xid+id\$	Reduce $T \rightarrow F$
(4)	0 2	T	xid+id\$	Shift
(5)	0 2 7	Tx	id+id\$	

At state 7, we have s5 operation for symbol id and s4 operation for symbol (. Since the current input symbol is id, we will go with the s5 operation. So, we will push id to the symbol column and 5 to the stack.

	Stack	Symbol	Input Buffer	Parsing Action
(1)	0		idxid+id\$	Shift
(2)	0 5	id	xid+id\$	Reduce $F \rightarrow id$
(3)	0 3	F	xid+id\$	Reduce $T \rightarrow F$
(4)	0 2	T	xid+id\$	Shift
(5)	0 2 7	T x	id+id\$	Shift
(6)	0 2 7 5	T x id	+id\$	

At state 5, we have an r6 operation for the symbol. So, we will reduce id by $F \rightarrow id$ and pop 5 out of the stack. We will also push 10 to the stack since for state 7 (now the symbol at the top of the stack), the GOTO function says to go to state 10 for F.

	Stack	Symbol	Input Buffer	Parsing Action
(1)	0		idxid+id\$	Shift
(2)	0 5	id	xid+id\$	Reduce $F \rightarrow id$
(3)	0 3	F	xid+id\$	Reduce $T \rightarrow F$
(4)	0 2	T	xid+id\$	Shift
(5)	0 2 7	T x	id+id\$	Shift
(6)	0 2 7 5	T x id	+id\$	Reduce $F \rightarrow id$

(7)	0 2 7 10	T x F	+id\$	
-----	----------	-------	-------	--

At state 10, we have r3 operation. So, we will reduce T x F by $T \rightarrow T \times F$ and pop all the states till T i.e. till 2 from the stack. We will also push 2 to the stack since for state 0 (now the symbol at the top of the stack), the GOTO function says to go to state 3 for F.

	Stack	Symbol	Input Buffer	Parsing Action
(1)	0		idxid+id\$	Shift
(2)	0 5	id	xid+id\$	Reduce $F \rightarrow id$
(3)	0 3	F	xid+id\$	Reduce $T \rightarrow F$
(4)	0 2	T	xid+id\$	Shift
(5)	0 2 7	T x	id+id\$	Shift
(6)	0 2 7 5	T x id	+id\$	Reduce $F \rightarrow id$
(7)	0 2 7 10	T x F	+id\$	Reduce $T \rightarrow T \times F$
(8)	0 2	T	+id\$	

At state 2, we have s7 operation for the x symbol and r2 operation for +,) and \$ symbol. Since the current input symbol is +, we will go with r2. So, we will reduce T by $E \rightarrow T$ and pop 2 from the stack. We will also push 1 to the stack since for state 0 (now the symbol at the top of the stack), the GOTO function says to go to state 1 for E.

	Stack	Symbol	Input Buffer	Parsing Action
(1)	0		idxid+id\$	Shift
(2)	0 5	id	xid+id\$	Reduce $F \rightarrow id$
(3)	0 3	F	xid+id\$	Reduce $T \rightarrow F$
(4)	0 2	T	xid+id\$	Shift
(5)	0 2 7	T x	id+id\$	Shift
(6)	0 2 7 5	T x id	+id\$	Reduce $F \rightarrow id$
(7)	0 2 7 10	T x F	+id\$	Reduce $T \rightarrow T \times F$

(8)	0 2	T	+id\$	Reduce $E \rightarrow T$
(9)	0 1	E	+id\$	

At state 1, we have an s6 operation for the + symbol. So, we will push + to the symbol column and 6 to the stack.

	Stack	Symbol	Input Buffer	Parsing Action
(1)	0		idxid+id\$	Shift
(2)	0 5	id	xid+id\$	Reduce $F \rightarrow id$
(3)	0 3	F	xid+id\$	Reduce $T \rightarrow F$
(4)	0 2	T	xid+id\$	Shift
(5)	0 2 7	T x	id+id\$	Shift
(6)	0 2 7 5	T x id	+id\$	Reduce $F \rightarrow id$
(7)	0 2 7 10	T x F	+id\$	Reduce $T \rightarrow T x F$
(8)	0 2	T	+id\$	Reduce $E \rightarrow T$
(9)	0 1	E	+id\$	Shift
(10)	0 1 6	E +	id\$	

At state 6, we have an s5 operation for the id symbol and an s4 operation for (symbol. Since the current input symbol is id, we will go with the s5 operation. So, we will push id to the symbol column and 5 to the stack.

	Stack	Symbol	Input Buffer	Parsing Action
(1)	0		idxid+id\$	Shift
(2)	0 5	id	xid+id\$	Reduce $F \rightarrow id$
(3)	0 3	F	xid+id\$	Reduce $T \rightarrow F$
(4)	0 2	T	xid+id\$	Shift
(5)	0 2 7	T x	id+id\$	Shift

(6)	0 2 7 5	T x id	+id\$	Reduce $F \rightarrow id$
(7)	0 2 7 10	T x F	+id\$	Reduce $T \rightarrow T x F$
(8)	0 2	T	+id\$	Reduce $E \rightarrow T$
(9)	0 1	E	+id\$	Shift
(10)	0 1 6	E +	id\$	Shift
(11)	0 1 6 5	E + id	\$	

At state 5, we have r6 operation. So, we will reduce id by $F \rightarrow id$ and pop 5 from the stack. We will also push 3 to the stack since for state 6 (now the symbol at the top of the stack), the GOTO function says to go to state 3 for F.

	Stack	Symbol	Input Buffer	Parsing Action
(1)	0		id+id\$	Shift
(2)	0 5	id	xid+id\$	Reduce $F \rightarrow id$
(3)	0 3	F	xid+id\$	Reduce $T \rightarrow F$
(4)	0 2	T	xid+id\$	Shift
(5)	0 2 7	T x	id+id\$	Shift
(6)	0 2 7 5	T x id	+id\$	Reduce $F \rightarrow id$
(7)	0 2 7 10	T x F	+id\$	Reduce $T \rightarrow T x F$
(8)	0 2	T	+id\$	Reduce $E \rightarrow T$
(9)	0 1	E	+id\$	Shift
(10)	0 1 6	E +	id\$	Shift
(11)	0 1 6 5	E + id	\$	Reduce $F \rightarrow id$
(12)	0 1 6 3	E + F	\$	

At state 3, we have an r4 operation. So, we will reduce F by $T \rightarrow F$ and pop 3 from the stack. We will also push 9 to the stack since for state 6 (now the symbol at the top of the stack), the GOTO function says to go to state 9 for T.

	Stack	Symbol	Input Buffer	Parsing Action
(1)	0		idxid+id\$	Shift
(2)	0 5	id	xid+id\$	Reduce $F \rightarrow id$
(3)	0 3	F	xid+id\$	Reduce $T \rightarrow F$
(4)	0 2	T	xid+id\$	Shift
(5)	0 2 7	T x	id+id\$	Shift
(6)	0 2 7 5	T x id	+id\$	Reduce $F \rightarrow id$
(7)	0 2 7 10	T x F	+id\$	Reduce $T \rightarrow T x F$
(8)	0 2	T	+id\$	Reduce $E \rightarrow T$
(9)	0 1	E	+id\$	Shift
(10)	0 1 6	E +	id\$	Shift
(11)	0 1 6 5	E + id	\$	Reduce $F \rightarrow id$
(12)	0 1 6 3	E + F	\$	Reduce $T \rightarrow F$
(13)	0 1 6 9	E + T	\$	

At state 9, we have s7 operation for x and r1 operation for +,) and \$. Since \$ is the current input symbol, we will go with r1. So, we will reduce $E + T$ by $E \rightarrow E + T$ and pop all the states till E, i.e., till 6 from the stack. And we will push 1 into the stack.

	Stack	Symbol	Input Buffer	Parsing Action
(1)	0		idxid+id\$	Shift
(2)	0 5	id	xid+id\$	Reduce $F \rightarrow id$
(3)	0 3	F	xid+id\$	Reduce $T \rightarrow F$
(4)	0 2	T	xid+id\$	Shift
(5)	0 2 7	T x	id+id\$	Shift

(6)	0 2 7 5	T x id	+id\$	Reduce $F \rightarrow id$
(7)	0 2 7 10	T x F	+id\$	Reduce $T \rightarrow T \times F$
(8)	0 2	T	+id\$	Reduce $E \rightarrow T$
(9)	0 1	E	+id\$	Shift
(10)	0 1 6	E +	id\$	Shift
(11)	0 1 6 5	E + id	\$	Reduce $F \rightarrow id$
(12)	0 1 6 3	E + F	\$	Reduce $T \rightarrow F$
(13)	0 1 6 9	E + T	\$	Reduce $E \rightarrow E + T$
(14)	0 1	E	\$	

At state 1, we have s6 operation for + and A i.e. accept for \$. Since the current input symbol is \$, the input string will be accepted, and the parsing will be completed.

	Stack	Symbol	Input Buffer	Parsing Action
(1)	0		idxid+id\$	Shift
(2)	0 5	id	xid+id\$	Reduce $F \rightarrow id$
(3)	0 3	F	xid+id\$	Reduce $T \rightarrow F$
(4)	0 2	T	xid+id\$	Shift
(5)	0 2 7	T x	id+id\$	Shift
(6)	0 2 7 5	T x id	+id\$	Reduce $F \rightarrow id$
(7)	0 2 7 10	T x F	+id\$	Reduce $T \rightarrow T \times F$
(8)	0 2	T	+id\$	Reduce $E \rightarrow T$
(9)	0 1	E	+id\$	Shift
(10)	0 1 6	E +	id\$	Shift
(11)	0 1 6 5	E + id	\$	Reduce $F \rightarrow id$

(12)	0 1 6 3	E + F	\$	Reduce $T \rightarrow F$
(13)	0 1 6 9	E + T	\$	Reduce $E \rightarrow E + T$
(14)	0 1	E	\$	Accept

• The following grammar will be used to illustrate LR parsing:

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow (E)$
6. $F \rightarrow id$

- The LR parsing table for this grammar:

State	Action						Goto		
	id	+	*	(	)	\$	E	T	F
0	S5			S4			1	2	3
1		S6				accept			
2		R2	S7		R2	R2			
3		R4	R4		R4	R4			
4	S5			S4			8	2	3
5		R6	R6		R6	R6			
6	S5			S4				9	3
7	S5			S4					10
8		S6			S11				
9		R1	S7		R1	R1			
10		R3	R3		R3	R3			
11		R5	R5		R5	R5			

R4 means reduce using rule 4; S6 means shift the next input symbol onto the stack and push state S_6 . Empty positions in the ACTION table indicate syntax errors.

- A trace of the parse of $id + id * id$ using the LR parsing algorithm:

Stack	Input	Action
0	$id + id * id \$$	Shift 5
0 id 5	$+ id * id \$$	Reduce 6 (use GOTO[0, F])
0 F 3	$+ id * id \$$	Reduce 4 (use GOTO[0, T])
0 T 2	$+ id * id \$$	Reduce 2 (use GOTO[0, E])
0 E 1	$+ id * id \$$	Shift 6
0 E 1+ 6	$id * id \$$	Shift 5
0 E 1+ 6 id 5	$* id \$$	Reduce 6 (use GOTO[6, F])
0 E 1+6 F 3	$* id \$$	Reduce 4 (use GOTO[6, T])
0 E 1+ 6 T 9	$* id \$$	Shift 7
0 E 1+ 6 T 9 *7	$id \$$	Shift 5
0 E 1+ 6 T 9*7 id 5	$\$$	Reduce 6 (use GOTO[7, F])
0 E1+ 6 T9*7 F10	$\$$	Reduce 3 (use GOTO[6, T])
0 E1+ 6 T 9	$\$$	Reduce 1 (use GOTO[0, E])
0 E1	$\$$	Accept

LR(1) Parser

state	action					goto		
	=	id	+	*	\$	G	E	T
0		s4				1	2	3
1					acc			
2	s5		s6					
3	r4		r4	s7				
4	r6		r6	r6	r2			
5		s10					8	9
6		s12						11
7		s13						
8			s14		r1			
9			r4	s15	r4			
10			r6	r6	r6			
11	r3		r3	s7				
12	r6		r6	r6				
13	r5		r5	r5				
14		s10						16
15		s17						
16			r3	s15	r3			
17			r5	r5	r5			

```

0 S' → G
1 G → E=E
2 G → id
3 E → E+T
4 E → T
5 T → T*id
6 T → id

```

id = id + id * id

Create [LR\(1\) parsers](https://ecomputernotes.com/compiler-design/lr-parsers) in your browser and see how they work.

<https://ecomputernotes.com/compiler-design/lr-parsers>

Verifying whether the string id * id + id is accepted by a given grammar using SLR parsing

Consider the SLR parsing table for the Grammar

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

Check whether the string id * id + id is accepted or not by using the SLR parsing table constructed in the example.

Solution

Initially, LR Parser in state 0.

Put \$ at the end of the string, i.e., id * id + id \$.

Stack	Input String	Reason
0	id * id + id	Action [0, id] = s5 ∴ Shift id and state 5

Stack	Input String	Reason
0 id 5	* id + id \$	Action [5,*] = r6. ∴ Reduce by $F \rightarrow id$. goto(0, F) = 3
0 F 3	* id + id \$	Action [3,*] = r4, Reduce by $T \rightarrow F$ goto(0, T) = 2
0 T 2	* id + id \$	Action [2,*] = s7, shift *, 7
0T2*7	id + id \$	Action [7, id] = s5, shift id, 5
0T2*7 id 5	+id \$	Action [5, +] = r6, Reduce by $F \rightarrow id$ goto(7, F) = 10
0T2*7 F 10	+id \$	Action [10, +] = r3S, Reduce by $T \rightarrow T * F$, goto(0, T) = 2
0 T 2	+id \$	Action [2, +] = r2, Reduce by $E \rightarrow T$ goto(0, E) = 1
0 E 1	+id \$	Action [1, +] = s6, Shift +, 6
0 E 1 + 6	id \$	Action [6, id] = s5, Shift id, 5.
0 E 1 + 6 id 5	\$	Action [5, \$] = s6, Reduce by $F \rightarrow id$, goto(6, F) = 3
0 E 1 + 6 F 3	\$	Action [3, \$] = r4, Reduce by $T \rightarrow F$, goto(6, F) = 9
0 E 1 + 6 T 9	\$	Action [9, \$] = r1, Reduce by $E \rightarrow E + T$, goto(0, E) = 1
0 E 1	\$	Action [1, \$] = accept

In the first row, we have 0 at the top of the stack, and the id symbol is to be processed. So, check Row (0) and column (id) i.e., Action [0, id] = s5, i.e., Shift id from string and state 5 onto stack.

In Second Row, Action [5, *] = r3 means reduce by production number 6 i.e., $F \rightarrow id$. Pop id from the stack and push F. Then see goto (0, F) = 3, so push 3 onto Stack. In Row seven, Action [10, +] = r3 i.e., reduce by production number 3 i.e., $T \rightarrow T * F$.

Pop T2 * 7F10 from the stack, push T into the stack. Then see goto [0, T] = 2. So, push 2 onto stack In Last Row Action [1, \$] = accept. That means the parser will accept the string.

Algorithm of SLR Parsing Table

There are two parts of the SLR Parsing table

- Action
- goto

Action and goto table can be filled using the following Algorithm –

Algorithm

Input– An Augmented Grammar G'

Output– SLR Parsing Table

Method

- Initially construct a set of items

$C = \{I_0, I_1, I_2 \dots I_n\}$ where C is a set of LR (0) items for Grammar.

- Parsing actions are based on each item or state I_i .

Various Actions are –

- If $A \rightarrow \alpha \cdot a \beta$ is in I_i and $\text{goto}(I_i, a) = I_j$ then set Action $[i, a] = \text{shift } j$.
- If $A \rightarrow \alpha \cdot$ is in I_i then set Action $[i, a]$ to "reduce $A \rightarrow \alpha$ " for all symbol a , where $a \in \text{FOLLOW}(A)$.
- If $S' \rightarrow S \cdot$ is in I_i then the entry in action table Action $[i, \$] = \text{accept}$.
- The goto part of the SLR table can be filled as– The goto transition for the state i is considered for non-terminals only. If $\text{goto}(I_i, A) = I_j$ then $\text{goto}[i, A] = j$
- All entries not defined by rules 2 and 3 are considered to be "error. "

Frequently Asked Questions

Define parser.

The parser is a compiler phase that accepts a token string as input and turns it into the matching intermediate representation using existing grammar. A parser is also known as a syntax analyzer.

Differentiate between top-down parsing and bottom-up parsing.

Top-Down Parsing is a parsing strategy that starts at the top of the parse tree and works its way down by applying grammatical rules. And Bottom-up Parsing is a parsing strategy that starts with the lowest level of the parse tree and works its way up using grammatical rules.

What is LR parsing?

LR Parsing is a type of bottom-up parsing. It is used to parse context-free grammar, mostly used by compilers. The LR parser reads the input from left to right and produces the rightmost derivation.

What are the types of LR parsing?

There are four types of LR parsing - LR(0), SLR(1), LALR(1) and CLR(1).

Define the ACTION and GOTO table.

The ACTION table specifies the grammar rules for implementing the input stream's current state and terminal in the parsing table. There are four cases used in the Action table. These are- Shift, Reduce, Accept, and Error. On the other hand, the GOTO table tells the next state to which the parsing should proceed.

Conclusion

In this chapter, we have studied the **LR parsing of syntax analysis**. We went through the concept thoroughly, discussing the block diagram, parsing table, steps, and an example of LR parsing.